



Nombre:

Grupo:

Pregunta 1 (2p): Contestar V/F las siguientes cuestiones, teniendo en cuenta que una respuesta incorrecta invalida una correcta.

[] Las siguientes sentencias definen y crean un vector estático de punteros a enteros.

```
int **p;  
*p = new int [100];
```

[] Jesús ha tenido una buena idea al utilizar una matriz dispersa para establecer una correspondencia V/F entre todos los aeropuertos comerciales del planeta que tienen vuelos directos.

[] Esta sentencia tiene sentido que se incluya en la primitiva que elimina el nodo siguiente al apuntado por p en una lista simplemente enlazada:

```
p->sig = p->sig->sig;
```

[] Esta sentencia es correcta usando STL y produce los resultados esperados:

```
list<int>l;
```

```
....
```

```
l.insert(l.begin() + 1, 100);
```

[] Para recorrer árboles sin recursividad, se debería utilizar una pila para realizar un recorrido en postorden y una cola para hacer un recorrido por niveles o anchura.

[] Una cola con prioridad puede ofrecer $O(n)$ en la inserción y $O(1)$ en el borrado.

[] Los agrupamientos secundarios ocurren cuando $h(x1) \neq h(x2)$ y luego compiten por las mismas posiciones.

[] Un quadtree es por definición, un árbol de orden 4 equilibrado en altura.

[] Si se introducen datos ordenados de forma ordenada ascendente en un AVL, el tipo de rotación que se realizaría será siempre 3 o 4.

[] No es posible albergar un millón de datos en un árbol B de orden 64 con tan solo 3 niveles (altura 2) pero sí con orden igual a 128.

Pregunta 2 (1.5p):

Definid una función de dispersión cuadrática para una tabla de dispersión de tamaño 13 para insertar (I) / borrar (B) los siguientes datos: I(33), I(21), I(7), I(27), I(0), I(22), I(13), I(14), B(0), B(22), I(18), B(7), B(33), I(2).

Dibujad paso a paso el proceso (se pueden agrupar varios si no hay colisiones) dejando claro el estado de cada casilla. Determinar el valor de λ máximo alcanzado y el dato *con mas colisiones*

Pregunta 3 (3p):

(a) Dada la definición de un árbol AVL<T> devolver el dato con el valor máximo albergado en dicho árbol en tiempo $O(\log n)$:

```
T AVL<T>::datoMax ();
```

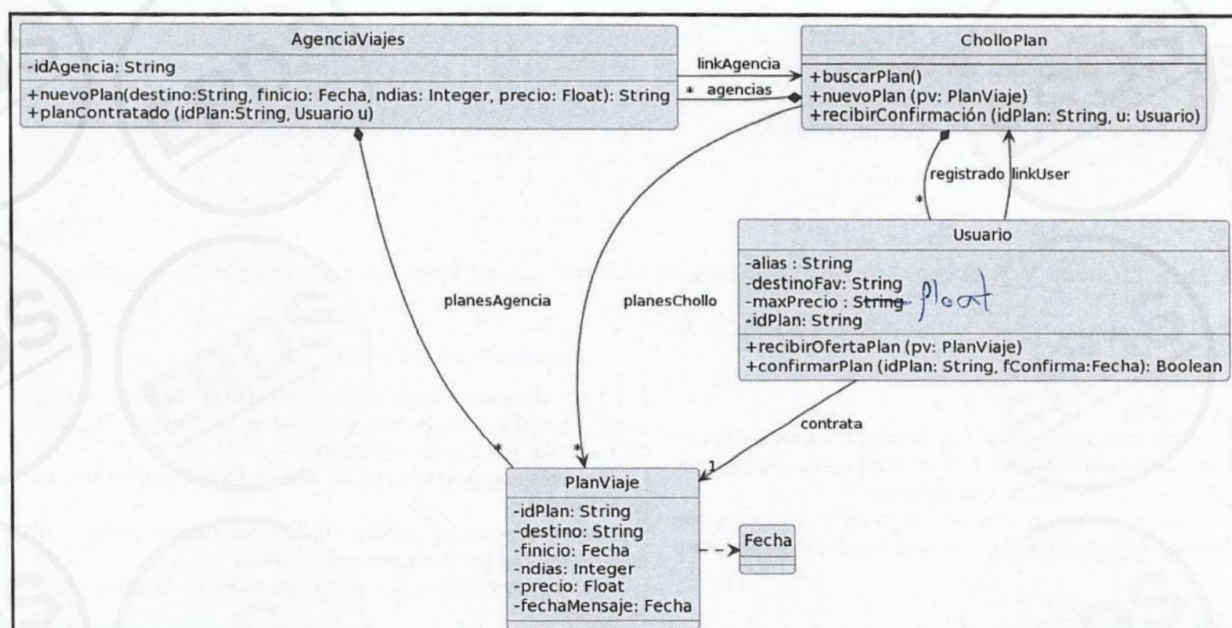
(a) Insertar un nuevo dato $ndato$ en una lista doblemente enlazada con datos ordeanados de forma ascendente para que manten-

ga este orden. La lista admite repetidos y se supone que ya está ordenada, por lo que no hay que comprobarlo de forma explícita, pero si se detectara que no lo está, se debería lanzar la excepción NON_SORTED_LIST(). (Aviso: no deben usarse iteradores).

```
void ListaDEnlazada<T>::insertaOrdenado (T &ndato);
```

Pregunta 4 (3.5p):

Un portal web llamada CholloPlan busca en diferentes webs o agencias de viajes chollos de última hora con un descuento increíble. Si eres usuario de la plataforma, te puede llegar un mensaje invitándote a hacer un viaje. Te informan del destino, el precio, fecha de salida y días y tienes 5 minutos para contestar. Si no se confirma pasa la vez y buscarán a otro usuario. El UML de la aplicación se describe a continuación:



PlanViaje: Es la clase que representa una sola oferta específica de viaje, en principio para dos viajeros. Tiene la información básica del viaje incluyendo el destino, el precio, la fecha de salida y el número de días. Cada vez que la oferta es enviada a un usuario, se actualiza el atributo *fechaMensaje* con la fecha del sistema. La oferta es creada por la agencia de viaje con *AgenciaViajes::nuevoPlan()*. El *IdPlan* lo crea la agencia de viajes de manera que los tres primeros caracteres coinciden con el *IdAgencia*. De este modo se puede conocer la identidad de la agencia que propone el viaje.

AgenciaViajes: es cada una de las agencias de viajes suscritas al proyecto. Cuando tiene una oferta de plan de viaje la añade a través de *planesAgencia* y avisa a *CholloPlan* a través de *linkAgencia* que hay una nueva oferta llamando a *CholloPlan::nuevoPlan()*. Cuando el viaje se ha contratado finalmente con *CholloPlan::recibirConfirmacion()*, entonces *CholloPlan* le avisa llamando a *planContratado()*. Esta función debe eliminar el plan de la relación *planesAgencia*. También pone en contacto al usuario con la agencia, pero el resto de operaciones para terminar de gestionar el viaje no se hace en este diseño.

Usuario: Representa a cada usuario cliente del sistema. Éste define su destino favorito y el precio máximo que está dispuesto a pagar. Si el destino está vacío, entonces le puede llegar cualquier destino posible, en caso contrario, sólo le llegan ofertas del destino marcado. Nunca le llegarán ofertas con precio superior al establecido. Cuando recibe una oferta que coincide con sus requisitos con *recibirOfertaPlan()*, le llega el plan para que lo mire en tan sólo 5 minutos. En ese tiempo la oferta actualiza su fecha demensaje con *PlanViaje::fechaMensaje* y en ese periodo de tiempo no se le puede dar a otro usuario. El id del último plan que se ha enviado se guarda en el atributo *idPlan*. Si no responde pasados esos 5 minutos, o lo hace después de ese tiempo, la oferta deja de ser válida y podrá pasar a otro usuario. Esta función se implementa como un mensaje en pantalla que muestra los atributos del plan de viaje (usar *cout* para simular el mismo efecto). El usuario no tiene mucho margen para pensar, si confirma finalmente se llama a la función *confirmarPlan()* indicando el id del plan que desea (ya guardado al llegar el mensaje) y la hora al al que responde (la fecha actual del sistema; suponed que la función *getTime():Fecha* devuelve la hora de sistema en cualquier parte del código). Si la diferencia entre la fecha del sistema y la guardada en *PlanViaje* es superior a 5 min, entonces la función devuelve false, sino devuelve true, y entonces mediante *linkUser* se llama a *CholloPlan::recibirConfirmaciónViaje()*.

CholloPlan: Es la clase que pone en contacto a clientes con los planes de las agencia de viajes. Mediante la función *buscarPlan()*, lo que se hace es recorrer a todos los usuarios de modo circular buscando si encaja alguno de los viajes disponibles y mandando la información. Esto ocurre en caso de que el usuario tenga alguna coincidencia de búsqueda considerando destino y precio con la función *Usuario::recibirOferta()*. Si hay varias opciones válidas para el usuario se manda cualquiera de ellas. En el momento que se envía una oferta, la fecha se apunta en *PlanViaje::fechaMensaje*. Esa oferta se bloquea durante 5 minutos, por lo que no se deben mandar ofertas si no ha pasado este tiempo. Si ya han pasado más de 5 minutos después de esa fecha, entonces se puede volver a mandar la oferta a otro usuario. La función *nuevoPlan()* es llamada por la agencia que crea una nueva oferta de viaje, y lo que debe hacerse es añadirla también a través de *planesChollo*. Finalmente, cuando se recibe la confirmación de que un usuario va a realizar el viaje con *Usuario::confirmarPlan()* se llama a *recibirConfirmacion()*. Entontes primero se elimina el plan de oferta de la relación *PlanesChollo* y luego se llama a *AgenciaViajes::planContrado()* para que la agencia también lo elimine de su cuenta particular.

NOTA: La clase Fecha se supone ya implementada y tiene disponible todos los operadores que se consideren necesarios (p.e. *operator-* que devuelve segundos, *operator<*, etc). Se supone también implementada la función *getTime():Fecha* que devuelve la hora del sistema incluyendo (a=año, m=mes, d=día, h=hora, m=minuto, s=segundo).